CrossEntropyLoss#

https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

Description:

This criterion computes the cross entropy loss between input logits and target. It is useful when training a classification problem with C classes. If provided, the optional argument weight should be a 1D Tensor assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set. The input is expected to contain the unnormalized logits for each class (which do not need to be positive or sum to 1, in general). input has to be a Tensor of size (C)(C)(C) for unbatched input, (minibatch,C)(minibatch, C)(minibatch,C) or (minibatch,C,d1,d2,...,dK)(minibatch, C, d_1, d_2, ..., d_K)(minibatch,C,d1,d2,...,dK) with K≥1K \geq 1K≥1 for the K-dimensional case. The last being useful for higher dimension inputs, such as computing cross entropy loss per-pixel for 2D images. The target that this criterion expects should contain either: Class indices in the range [0,C)[0, C)[0,C) where CCC is the number of classes; if ignore_index is specified, this loss also accepts this class index (this index may not necessarily be in the class range). The unreduced (i.e. with reduction set to 'none') loss for this case can be described as:

Function Signature

```
class torch.nn.CrossEntropyLoss(weight=None,
size_average=None, ignore_index=-100, reduce=None,
reduction='mean', label_smoothing=0.0)[source]#

Parameters

Shape:

forward(input, target)[source]#

Return type
```

Returns:

Tensor

Code Examples

```
>>> # Example of target with class indices
>>> loss = nn.CrossEntropyLoss()
>>> input = torch.randn(3, 5, requires_grad=True)
>>> target = torch.empty(3, dtype=torch.long).random_(5)
>>> output = loss(input, target)
>>> output.backward()
>>>
>>> # Example of target with class probabilities
>>> input = torch.randn(3, 5, requires_grad=True)
>>> target = torch.randn(3, 5).softmax(dim=1)
>>> output = loss(input, target)
>>> output.backward()
```

```
>>> # Example of target with incorrectly specified class
probabilities
>>> loss = nn.CrossEntropyLoss()
>>> torch.manual_seed(283)
>>> input = torch.randn(3, 5, requires_grad=True)
>>> target = torch.randn(3, 5)
>>> # Provided target class probabilities are not in range [0,1]
>>> target
tensor([[ 0.7105,  0.4446,  2.0297,  0.2671, -0.6075],
        [-1.0496, -0.2753, -0.3586,  0.9270,  1.0027],
        [ 0.7551,  0.1003,  1.3468, -0.3581, -0.9569]])
>>> # Provided target class probabilities do not sum to 1
>>> target.sum(axis=1)
tensor([2.8444, 0.2462, 0.8873])
>>> # No error message and possible misleading loss value
>>> loss(input, target).item()
4.6379876136779785
>>>
>>> # Example of target with correctly specified class
probabilities
>>> # Use .softmax() to ensure true probability distribution
>>> target_new = target.softmax(dim=1)
>>> # New target class probabilities all in range [0,1]
>>> target_new
tensor([[0.1559, 0.1195, 0.5830, 0.1000, 0.0417],
        [0.0496, 0.1075, 0.0990, 0.3579, 0.3860],
        [0.2607, 0.1355, 0.4711, 0.0856, 0.0471]])
>>> # New target class probabilities sum to 1
>>> target_new.sum(axis=1)
tensor([1.0000, 1.0000, 1.0000])
>>> loss(input, target_new).item()
2.55349063873291
```

Notes & Warnings

NOTE Note The performance of this criterion is generally better when target contains class indices, as this allows for optimized computation. Consider providing target as class probabilities only when a single class label per minibatch item is too restrictive.

NOTE Note When target contains class probabilities, it should consist of soft labels—that is, each target entry should represent a probability distribution over the possible classes for a given data sample, with individual probabilities between [0,1] and the total distribution summing to 1. This is why the softmax() function is applied to the target in the class probabilities example above. PyTorch does not validate whether the values provided in target lie in the range [0,1] or whether the distribution of each data sample sums to 1. No warning will be raised and it is the user's responsibility to ensure that target contains valid probability distributions. Providing arbitrary values may yield misleading loss values and unstable gradients during training.

Detailed Documentation

Documentation

This criterion computes the cross entropy loss between input logits and target.

It is useful when training a classification problem with C classes. If provided, the optional argument weight should be a 1D Tensor assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set.

The input is expected to contain the unnormalized logits for each class (which do not need to be positive or sum to 1, in general). input has to be a Tensor of size (C)(C)(C) for unbatched input, (minibatch,C)(minibatch, C)(minibatch,C) or (minibatch,C,d1,d2,...,dK)(minibatch, C, d_1, d_2, ..., d_K)(minibatch,C,d1,d2,...,dK) with K≥1K \geq 1K≥1 for the K-dimensional case. The last being useful for higher dimension inputs, such as computing cross entropy loss per-pixel for 2D images.

The target that this criterion expects should contain either:

Class indices in the range [0,C)[0, C)[0,C) where CCC is the number of classes; if ignore_index is specified, this loss also accepts this class index (this index may not necessarily be in the class range). The unreduced (i.e. with reduction set to 'none') loss for this case can be described as:

where xxx is the input, yyy is the target, www is the weight, CCC is the number of classes, and NNN spans the minibatch dimension as well as d1,...,dkd_1, ..., d_kd1 ,...,dk for the K-dimensional case. If reduction is not 'none' (default 'mean'), then

Note that this case is equivalent to applying LogSoftmax on an input, followed by NLLLoss.

Probabilities for each class; useful when labels beyond a single class per minibatch item are required, such as for blended labels, label smoothing, etc. The unreduced (i.e. with reduction set to 'none') loss for this case can be described as:

where xxx is the input, yyy is the target, www is the weight, CCC is the number of classes, and NNN spans the minibatch dimension as well as d1,...,dkd_1, ..., d_kd1 ,...,dk for the K-dimensional case. If reduction is not 'none' (default 'mean'), then

The performance of this criterion is generally better when target contains class indices, as this allows for optimized computation. Consider providing target as class probabilities only when a single class label per minibatch item is too restrictive.

weight (Tensor, optional) – a manual rescaling weight given to each class. If given, has to be a Tensor of size C.

size_average (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field size_average is set to False, the losses are instead summed for each minibatch. Ignored when reduce is False. Default: True

ignore_index (int, optional) – Specifies a target value that is ignored and does not contribute to the input gradient. When size_average is True, the loss is averaged over non-ignored targets. Note that ignore_index is only applicable when the target contains class indices.

reduce (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on size_average. When reduce is False, returns a loss per batch element instead and ignores size_average. Default: True

reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the weighted mean of the output is taken, 'sum': the output will be summed. Note: size_average and reduce are in the process of being deprecated, and in the meantime, specifying either of those two args will override reduction. Default: 'mean'

label_smoothing (float, optional) – A float in [0.0, 1.0]. Specifies the amount of smoothing when computing the loss, where 0.0 means no smoothing. The targets become a mixture of the original ground truth and a uniform distribution as described in

Rethinking the Inception Architecture for Computer Vision. Default: 0.00.00.0.

Input: Shape (C)(C)(C), (N,C)(N, C)(N,C) or (N,C,d1,d2,...,dK)(N, C, d_1, d_2, ..., d_K) (N,C,d1,d2,...,dK) with K≥1K \geq 1K≥1 in the case of K-dimensional loss.

Target: If containing class indices, shape ()()(), (N)(N)(N) or (N,d1,d2,...,dK)(N, d_1, d_2, ..., d_K)(N,d1,d2,...,dK) with K≥1K \geq 1K≥1 in the case of K-dimensional loss where each value should be between [0,C)[0, C)[0,C). The target data type is required to be long when using class indices. If containing class probabilities, the target must be the same shape input, and each value should be between [0,1][0, 1][0,1]. This means the target data type is required to be float when using class probabilities. Note that PyTorch does not strictly enforce probability constraints on the class probabilities and that it is the user's responsibility to ensure target contains valid probability distributions (see below examples section for more details).

Output: If reduction is 'none', shape ()()(), (N)(N)(N) or (N,d1,d2,...,dK)(N, d_1, d_2, ..., d_K)(N,d1,d2,...,dK) with K≥1K \geq 1K≥1 in the case of K-dimensional loss, depending on the shape of the input. Otherwise, scalar.

When target contains class probabilities, it should consist of soft labels—that is, each target entry should represent a probability distribution over the possible classes for a given data sample, with individual probabilities between [0,1] and the total distribution summing to 1. This is why the softmax() function is applied to the target in the class probabilities example above.

PyTorch does not validate whether the values provided in target lie in the range [0,1] or whether the distribution of each data sample sums to 1. No warning will be raised and it is the user's responsibility to ensure that target contains valid probability distributions. Providing arbitrary values may yield misleading loss values and unstable gradients during training.

Runs the forward pass.